PIPM: Partial and Incremental Page Migration for Multi-host CXL Disaggregated Shared Memory

Gangqi Huang
Computer Science Engineering,
University of California, Santa Cruz
Santa Cruz, California, USA
ghuang49@ucsc.edu

Heiner Litz
Computer Science Engineering,
University of California, Santa Cruz
Santa Cruz, California, USA
hlitz@ucsc.edu

Yuanchao Xu
Computer Science Engineering,
University of California, Santa Cruz
Santa Cruz, California, USA
yxu314@ucsc.edu

Abstract

The emerging Compute Express Link (CXL) interconnect supports multi-host cache-coherent disaggregated shared memory (CXL-DSM). However, existing page migration approaches, designed primarily for single-host systems, are inefficient in multi-host CXL-DSM scenarios. To address this, we propose Partial and Incremental Page Migration (PIPM), a hardware-based solution that transparently leverages host-side local memory. PIPM is co-designed with the CXL multi-host coherence protocol, enabling coherent access to data residing in local DRAM. To overcome limitations of existing migration methods, PIPM supports fine-grained data migration and integrates hardware-based monitoring and decision-making mechanisms to optimize data placement. Evaluation results demonstrate that PIPM delivers performance improvements of up to 2.54× (1.86× on average) over the default multi-host CXL-DSM configuration.

CCS Concepts: • **Computer systems organization → n-tier architectures**; **Heterogeneous (hybrid) systems**; • **Hardware → Memory and dense storage**.

Keywords: Distributed Shared Memory, Disaggregated Memory, Page Migration, Cache Coherency, Compute eXpress Link

ACM Reference Format:
Gangqi Huang, Heiner Litz, and Yuanchao Xu. 2026. PIPM: Partial and Incremental Page Migration for Multi-host CXL Disaggregated Shared Memory. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26), March 22–26, 2026, Pittsburgh, PA, USA.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3779212.3790203



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '26, Pittsburgh, PA, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2359-9/2026/03
https://doi.org/10.1145/3779212.3790203

1 Introduction

Emerging applications in AI [14], databases [33], and big-data analytics [83] increasingly demand higher memory capacity, greater bandwidth, and lower costs [3, 5, 7, 13, 23, 25, 26, 40, 41, 43, 45–47, 55, 62, 64, 68, 70, 74, 86, 93]. With the slowdown of DRAM technology scaling [48, 57], architects have turned to Compute Express Link (CXL) for flexible, disaggregated shared memory (CXL-DSM), significantly improving efficiency and reducing DRAM costs [20, 48, 55, 60]. The latest CXL standards (CXL 3.x) further support coherent multi-host shared memory, enabling dynamic compute resource allocation and flexible memory partitioning, enhancing throughput and cost efficiency [20, 33, 53, 94, 95].

Recent research highlights substantial benefits of multi-host CXL-DSM across various applications [4, 6, 33, 37, 53, 82, 88, 94]. For example, HydraRPC uses CXL-DSM to improve RPC scalability [53], CXLfork reduces local memory consumption by 87% on average for cross-host process cloning [6], Tigon achieves an average 2.5× throughput improvement for databases compared to configurations without CXL-DSM [33], and PolarCXLMem [94] shows an up to 154.4% performance improvement compared to RDMA-based cloud databases [2, 21, 95].

Despite its potential, CXL-based system performance is often limited by the high latency of remote CXL memory accesses [48, 55, 94], which are typically two to three times slower than local DRAM accesses upon LLC misses [48]. A common solution is page migration [45, 46, 49, 50, 55, 70, 85, 90]: pages identified as frequently accessed by a host are migrated from CXL memory to a host's local memory, converting subsequent remote accesses into low-latency local accesses.

However, existing page migration schemes designed for single-host CXL disaggregated memory are ineffective in multi-host CXL-DSM for two reasons: **(1) Local gain, global pain.** In single-host systems, migrating a hot page to local DRAM is strictly beneficial, assuming sufficient local memory capacity is available. In a multi-host CXL-DSM, however, moving a hot page from shared CXL memory to one host's local DRAM may harm overall performance, outweighing the local benefit. To preserve coherence and consistency, the migrated page needs to become non-cacheable for all other

hosts. As a result, remote accesses incur extra hops, round-trips, and address remapping overheads, significantly increasing latency for other hosts accessing the page. **(2) Poor Migration Scalability.** The side-effects of page migration in multi-host CXL-DSM systems pose significant challenges for supporting efficient and timely migration. However, migration overheads grow significantly as page migration is no longer entirely local but instead requires coordination across hosts, including CXL RPCs [53], per-host page-table updates and TLB shootdowns.

To address these challenges, we propose **Partial and Incremental Page Migration (PIPM)** for multi-host CXL-DSM. **Partial Migration:** Instead of migrating entire pages into local memory or retaining them fully in CXL memory, PIPM selectively migrates only those cache blocks frequently accessed by a host into its local memory, while keeping less-frequently or remotely accessed blocks in CXL memory. This selective strategy differentiates local from inter-host access patterns at a fine granularity, effectively resolving the "local-gain, global-pain" issue. Moreover, by maintaining two possible destinations for cache blocks, PIPM significantly reduces migration management overheads, such as page-table updates and TLB invalidations. **Incremental Migration:** Rather than explicitly migrating entire pages, which incurs substantial data-transfer overhead, PIPM leverages intrinsic memory accesses of programs to migrate cache blocks incrementally and selectively. Specifically, PIPM determines whether to incrementally migrate cache blocks from CXL memory into the requester host's local memory or back to CXL memory during cache coherence request handling. Consequently, incremental migration involves no additional data transfers beyond regular cache-fill and eviction operations. The partial migration policy identifies cache blocks and target hosts without initiating immediate data transfers, relying entirely on incremental migration for data movement. Together, these techniques enable PIPM to systematically address the previously identified challenges.

We develop architectural support for PIPM, including a majority-vote migration policy, a two-level hardware remapping table, and PIPM-coherency to effectively enable partial and incremental page migration. We evaluate our technique using the Championship simulator [1, 24]. PIPM achieves an average speedup of 1.86× on multi-host CXL-DSM systems and surpasses six state-of-the-art methods.

Overall, this paper makes the following contributions:

1. Qualitatively and quantitatively identifies the challenges of page migration in multi-host CXL-DSM.
2. Introduces partial and incremental page migration to systematically address these challenges.
3. Presents an architectural design that effectively and efficiently supports partial and incremental page migration.
4. Provides a comprehensive evaluation demonstrating the effectiveness of PIPM.

2 Background

2.1 CXL Disaggregated Shared Memory

The CXL 3.0 standard introduces CXL Disaggregated Shared Memory (CXL-DSM) [10, 35, 59–61, 71, 72], allowing a pool of CXL memory to be shared coherently across multiple hosts. This contrasts to prior versions of CXL in which the CXL pool had to be statically partitioned and each partition assigned to one particular host. Also note that CXL 3.0 only allows coherent sharing of the CXL memory pool, while each host's local memory remains invisible to other hosts.

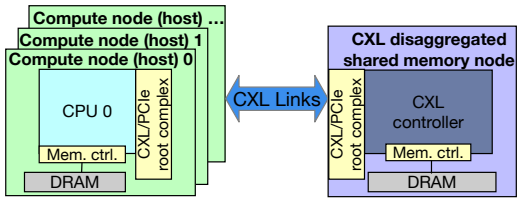


Figure 1. Multi-host CXL-DSM architecture.

Figure 1 illustrates a multi-host CXL-DSM architecture comprising multiple compute nodes (hosts) connected to a CXL memory node. Each host or memory node integrates a CXL/PCIe Root Complex (RC) that issues and receives messages over CXL links. The memory node contains a CXL/PCIe RC, a CXL controller, and one or more memory controllers connected to multiple memory devices [19, 79, 94]. The CXL controller manages connections and access to the attached memory. By allowing multiple hosts to attach concurrently, CXL-DSM enables cache-coherent data sharing and collaborative computation across hosts. Optional CXL switches [48, 94, 94] can be inserted between hosts and devices to realize even larger multi-host systems.

2.2 CXL-DSM Cache Coherence over CXL.mem

CXL-DSM supports multi-host cache coherence [20, 36] using a hierarchical, directory-based MESI protocol. Figure 2 illustrates a simplified organization of the CXL coherence architecture comprising two cooperating components: (i) a per-processor local coherence directory and (ii) a device coherence directory on the CXL memory node. The per-processor directory records the local coherence state and the core IDs for each cache line resident in that processor's cache (including both local memory and CXL memory). The device coherence directory records the coherence state and the processor IDs for each CXL memory cache line that reside in processors' caches. Throughout this paper, without loss of generality, we assume that each host contains only one processor to simplify the description.

The coherent CXL memory access workflow proceeds as follows. A request is first sent to the local coherence directory to determine whether the requesting processor's cache holds the most recent version of the target cache line ❶. On a *local*

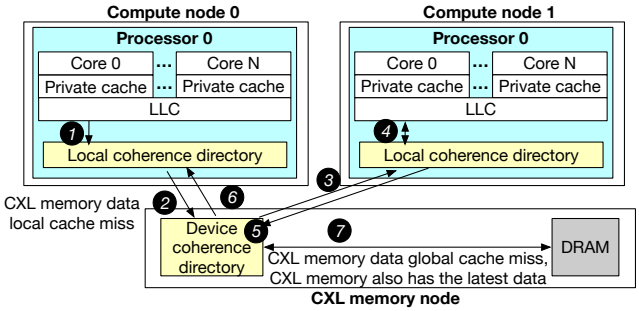


Figure 2. Coherence design of CXL-DSM.

cache miss, the request is forwarded to the device coherence directory on the CXL memory node ❷. If the device directory identifies a processor as the current owner (i.e., in M state), the request is forwarded to that processor ❸ to retrieve the latest data ❹. The data is then returned to the requester, and both the device directory state and the requester's local directory state are updated ❺❻. Conversely, if no processor holds the data (i.e., in I state), or if both CXL memory and a processor's cache hold clean copies (i.e., in S state), the request is satisfied directly from the CXL memory, and the directory states are updated accordingly ❼.

3 Motivation

3.1 Detailed Analysis of Multi-Host Migration

The CXL 3.1 standard and beyond [20] introduces the concept of *Global Integrated Memory (GIM)* [20, 42], allowing each host to expose part of its local memory into a global, unified memory address space. A host's page table can map a page that resides in its own local memory, in another host's local memory, or in CXL memory, thus allowing page migration between local memory and CXL memory. Inter-host accesses to another host's local memory [42, 97] are *non-cacheable to the requester host* [20, 38, 42, 97], thus always need to be routed through CXL root complexes, CXL links, and optional CXL switches. We present a simplified design consistent with CXL 3.1 to illustrate the page migration and access workflows.

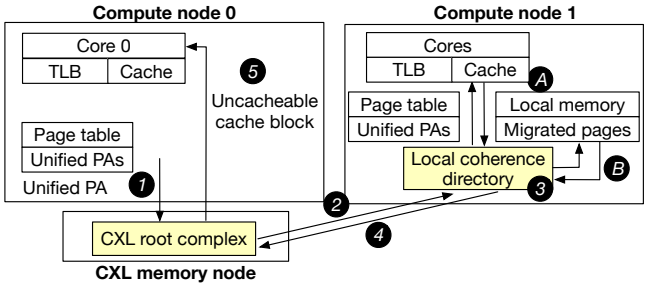


Figure 3. Workflow of accessing of migrated pages.

Workflow of inter-host access of migrated pages. Figure 3 ❶-❺ illustrates how a host accesses a page that has been migrated to another host's local memory. The local host processor first obtains the unified physical address (PA) from the TLB and page table and forwards it to the CXL Root Complex at the CXL memory node ❶. The root complex routes the request to the owning host indicated by the unified PA ❷. At the owning host, the local coherence directory is used to determine whether the most recent value resides in cache or in memory ❸; the data is then fetched into the owning node's LLC and returned to the CXL memory node ❹. The returned block, which is treated as non-cacheable at the requester host, is then delivered to the requester core. Serving this read miss requires a 4-hop traversal for the non-cacheable access. However, when the data resides in CXL memory, accesses are cacheable, which requires only two hops.

Take-away #1: In a multi-host CXL-DSM system, inter-host accesses to a migrated page are non-cacheable and require four hops. By contrast, accesses to CXL memory are cacheable and require at most two hops.

Workflow of local access of migrated pages. The non-cacheable access design increases the complexity of inter-host accesses but simplifies local accesses by eliminating coherence checks at the CXL memory node. As shown in Figure 3 ⒶⒷ, when a LLC read miss occurs, the unified PA is used to consult the local coherence directory to determine whether any cache within the host holds the most recent valid copy. If not, the request is served from local memory. As all inter-host accesses are non-cacheable, the design omits coherence probes to caches on other hosts, streamlining local-memory access.

Workflow of page migration. Page migration modifies a page's unified PA, which necessitates page table updates and TLB invalidations across all hosts. Each host uses its reserved page table to locate the process page tables that use the previous unified PA of the migrating page, and updates those entries to the new unified PA. Compared to single-host CXL disaggregated memory systems, this operation incurs a higher overhead in multi-host CXL-DSM because it requires broadcasting CXL RPCs [53] and performing more page table updates and TLB invalidations.

3.2 Quantitative Evaluation of Multi-Host Migration

Although recent research has investigated page migration policies and overhead optimization for single-host CXL disaggregated memory systems [34, 45, 55, 78, 90, 92, 96, 99, 100], these approaches are ineffective in multi-host CXL-DSM due to their lack of awareness regarding the side effects of page migration and the poor migration scalability from higher demands in multi-host environments.

We evaluate existing page migration policies, originally designed for single-host CXL disaggregated memory systems,

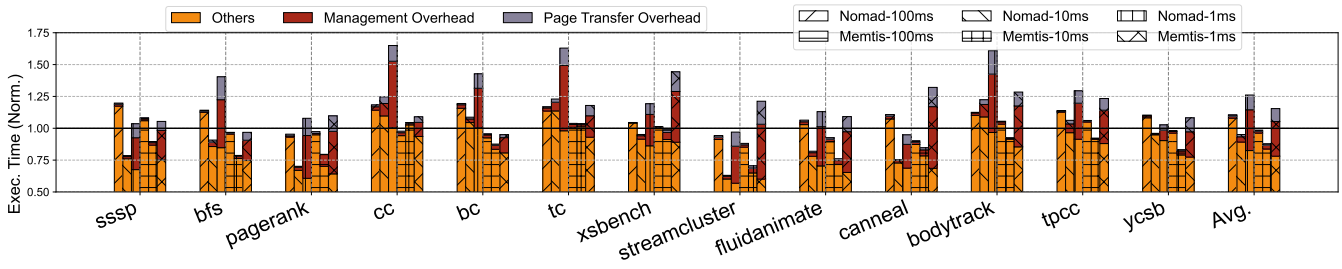


Figure 4. Performance breakdown with different page migration intervals, normalized to the no migration baseline.

in a multi-host CXL-DSM environment to quantitatively assess the performance impact of the two limitations. Existing page migration policies can be broadly classified into recency-based [32, 34, 54, 55, 90] and frequency-based methods [45, 68, 76]. Specifically, we evaluate two state-of-the-art (SOTA) policies, Nomad (a recency-based method) [90] and Memtis (a frequency-based method) [45], using the Championship simulator [1, 24] configured as a four-host CXL-DSM system, with each host containing a single-socket CPU. Our evaluation employs memory-intensive benchmarks from prior studies, drawn from the GAP [9], PARSEC 3.0 [98], XSBench [81], YCSB [18, 84] and TPC-C [84] benchmark suites. Detailed evaluation settings are described in Section 5.1.

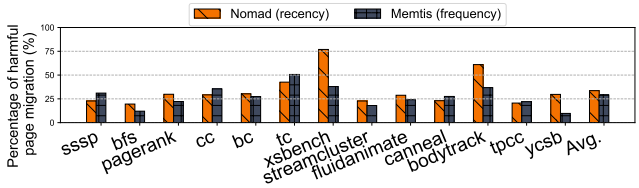


Figure 5. Percentage of harmful page migration.

3.2.1 Side Effects of Page Migration on multi-host CXL-DSM.

We evaluated the percentage of harmful page migrations to understand the impact of neglecting the side effects of page migration in multi-host CXL-DSM. After a page is migrated from CXL memory to a host's local memory, subsequent accesses from that host transition from remote CXL memory access to local memory access. However, other hosts experience increased latency and non-cacheable accesses when referencing the migrated page compared to scenarios without migration. Thus, we define a page migration as harmful if it increases the overall execution time. We report the percentage of harmful page migrations observed in existing studies.

Figure 5 illustrates the percentage of harmful page migrations. On average, Nomad and Memtis exhibit 34% and 29% harmful migrations, respectively. These migrations negatively impact overall performance by increasing total execution time; refraining from performing such migrations would enhance performance. The increase in execution time arises because migrations convert accesses from other hosts into inter-host non-cacheable accesses, underscoring the importance of accounting for side effects in multi-host CXL-DSM page migration algorithms.

Take-away #2: Neglecting side effects in multi-host CXL-DSM page migration, existing migration techniques, such as Nomad and Memtis, result in 34% and 29% performance-degrading page migrations, respectively.

3.2.2 Poor Migration Scalability.

Existing page migration techniques tailored for single-host systems generally adopt relatively long migration intervals (10 ms [68, 100] to a few seconds [32, 34, 55]) to balance migration overhead and performance benefits. However, the side-effects of page migration in multi-host CXL-DSM systems necessitate more efficient and timely migration mechanisms. We further conduct evaluations to quantitatively investigate: (1) whether multi-host CXL-DSM systems benefit from shorter page migration intervals (i.e., more timely and aggressive migration), and (2) the overhead breakdown associated with varying intervals. We report the performance breakdown, including page transfer overhead (data transfers incurred by migration), management overhead (e.g., page table updates and TLB invalidations), and other overheads.

Figure 4 presents the performance breakdown across three different page migration intervals (100 ms, 10 ms and 1ms), normalized against a no-migration baseline. The two state-of-the-art single-host migration methods show limited effectiveness in multi-host scenarios at the long interval (100 ms): Nomad increases execution time by 10.5% on average, while Memtis reduces it by only 1.4%. When adopting a shorter interval (10 ms) for more frequent page migration, execution time decreases by 4.8% and 12.2% on average. However, at the 1 ms interval, Nomad and Memtis increase execution time by 26.1% and 15.4% on average, respectively, due to the increased management overhead and page transfers.

Take-away #3: Multi-host CXL-DSM systems require shorter migration intervals to effectively capture page accesses from multiple hosts.

Take-away #4: At shorter intervals, page migration overhead becomes the dominant source of overhead, requiring efficient page migration.

3.3 Other Related Work

Several recent studies have investigated page migration in single-host CXL-disaggregated memory systems; however, their contributions does not address the previously discussed challenges associated with multi-host CXL-DSM page migration. They are orthogonal with the objectives pursued by our work. Specifically, Neomem [100] and M5 [78] offload hotness detection to the CXL memory side to facilitate efficient, low-latency access tracking. Colloid [85] balances memory placement between local and remote memory to minimize overall latency. Alto and Soar [50] employ MLP-aware policies to determine and dynamically adjust initial memory allocations across local and remote memory.

Intel Flat Mode [65, 99] is a recently introduced hardware-tiering technology designed for single-host CXL-disaggregated memory systems. Under this scheme, when a host accesses a cache block residing in CXL memory, the block is transparently swapped with a corresponding block in the host's local memory. However, Intel Flat Mode is incompatible with multi-host CXL-DSM. First, swapping memory lines between local memory and CXL memory switches the coherence domain between cache-coherent CXL-DSM and non-cacheable local memory, thereby violating coherence requirements. Second, Intel Flat Mode employs a static one-to-one mapping between CXL memory and local DRAM [99], which is impractical in multi-host environments where each host has distinct local DRAM regions. In our evaluation, we implement an Intel Flat Mode-like baseline (referred to as **HW-static**), utilizing parts of our design, to allow comparisons with hardware-tiering approaches.

4 Design

4.1 Overview

Based on the quantitative and qualitative analysis in Section 3, an effective and efficient page migration for multi-host CXL-DSM should consider the side effects of migrating data from CXL memory to local memory and reduce page migration overhead.

We attribute the inefficiency of existing single-host page migration methods to their **single-destination and rigid per-page migration**. Specifically, even when certain cache blocks within a page are frequently accessed by one host and other blocks are rarely accessed or predominantly accessed by other hosts, existing strategies either fully migrate the entire page or retain it entirely within CXL memory. This strategy fails to exploit optimization opportunities by treating different cache blocks within the same page separately (i.e., selectively migrating cache blocks). Additionally, per-page migration at low migration intervals incurs substantial overhead due to both management operations and data transfer costs.

We propose **Partial and Incremental Page Migration (PIPM)** for multi-host CXL-DSM. **Partial Migration** selectively migrates only frequently accessed cache blocks of a page to a host's local memory while leaving less-used blocks in CXL memory. This approach differentiates local and inter-host access patterns at fine granularity, mitigating side effects associated with per-page migration and significantly reducing management overhead (e.g., page-table updates and TLB invalidations). **Incremental Migration** leverages intrinsic memory accesses to incrementally migrate cache blocks upon cache eviction or writeback, avoiding explicit whole-page migrations and associated data-transfer overheads. Collectively, PIPM effectively addresses the previously identified challenges in multi-host memory management.

We develop architectural support to effectively and efficiently enable PIPM, facilitating transparent partial and incremental migration without requiring software modifications. As illustrated in Figure 6, our design introduces a per-host **Local Remapping Table** and a **Global Remapping Table** located on CXL memory to track pages undergoing partial migration. Specifically, the global remapping table records the migration destination host ID for each CXL-DSM page, while the local remapping table on each host stores the physical address mappings of CXL-DSM pages that migrate to the local memory of that host. We propose a **PIPM Majority-vote Migration Policy** that aggregates page-access information across multiple hosts, enabling globally optimized decisions regarding the necessity and placement of partially migrated pages. To ensure coherent access to partially migrated pages, we design the **PIPM Coherence** to incorporate partially migrated pages into the coherence domain, permitting incremental migration and cacheable access by other hosts.

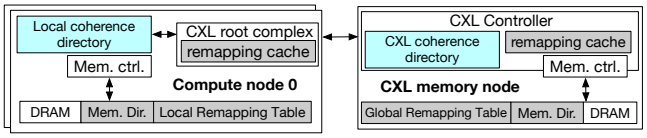


Figure 6. PIPM design overview.

4.2 PIPM Migration Policy

Existing page migration policies [15, 32, 93] are ineffective in multi-host CXL-DSM environments due to their neglect of migration side effects (**Takeaways #1 and #2**). To address this, PIPM introduces a hardware-based majority-vote migration policy inspired by the Boyer-Moore algorithm [11], enabling globally optimized decisions regarding the necessity and placement of partially migrated pages.

The intuition behind PIPM majority-vote migration policy is that partial migration is initiated only when the number of page accesses from a single host exceeds the combined accesses from all other hosts by a predefined threshold. It is important to note that initiating partial migration only involves updating the local and global remapping tables; thus, no page-table updates or TLB invalidations are required. The partial migration step only identifies the host to which cache blocks should be migrated, without triggering immediate data transfers.

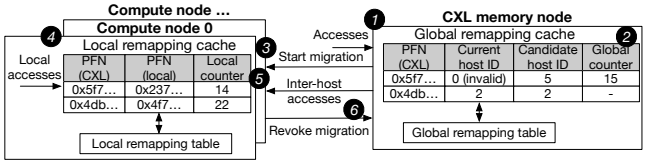


Figure 7. Partial migration workflow.

Figure 7 illustrates the architectural components designed to support the PIPM migration policy. The global remapping cache records recently accessed CXL pages and is backed by an in-memory global remapping table. Each entry in the global remapping table records metadata for a CXL-DSM page, comprising a 5-bit current host ID, a 5-bit candidate host ID, and a 6-bit global counter. The local remapping table of each host only tracks pages partially migrated to that host. Each entry in the local remapping table contains a 28-bit PFN (indexing 1TB local DRAM) referring to as the page's PFN in local memory, and a 4-bit local counter.

The global and local counters implement the PIPM majority-vote migration policy for initiating and revoking partial migration, as described below: The global counter tracks whether a particular host (indicated by the candidate host ID) has more accesses than all other hosts to issue partial migration. Specifically, the global counter is incremented by one when the access originates from the candidate host and decremented by one when accessed by other hosts. When the global counter reaches zero, the next host to access the page updates the candidate host ID ❶. If the global counter reaches a predefined partial migration threshold ❷, partial migration of this page is initiated by creating an entry in the candidate host's local remapping table. The local PFN for this entry, allocated by the host's OS/hypervisor, identifies the location where partially migrated data from CXL memory is stored, and the entry's local counter is initialized to the migration threshold ❸. After a page has been partially migrated, its current host ID is set to that host's ID.

The local counter, stored in each host's local remapping table, records local accesses to partially migrated pages since local accesses bypass the global counter maintained at the CXL memory node ❹. Also, inter-host accesses decrement the local counter for that page ❺. If the local counter of a partially migrated page reaches zero, partial migration for the page is revoked by migrating all cache blocks from local memory back to their original CXL memory location, removing the corresponding entry from the local remapping table, and resetting the current host ID in the corresponding global remapping table entry ❻.

4.3 PIPM Coherence and Incremental Migration Design

In existing multi-host CXL-DSM systems, only the hosts' caches and CXL memory are within the coherence domain. The hosts' local memory lies outside this coherence domain, precluding our proposed PIPM approach, as partially migrated cache blocks in local memory cannot be accessed coherently and cacheably.

4.3.1 Naive Coherence Solution. A straightforward solution is to introduce a 1-bit in-memory state for each cache block in the CXL memory to track partially migrated data[1]. This state indicates whether the associated cache block holds the most recent version; if it does not, the request will be redirected to the alternative memory (either local or CXL) to retrieve the latest data. However, this approach is inefficient for multi-host CXL-DSM because existing coherence protocols require completing a coherence state check for all caches in the CXL memory node and initiating a memory access from the CXL memory node—even if the latest version resides in local memory. This complexity arises from the potential for other hosts' caches to hold the latest version due to cacheable accesses.

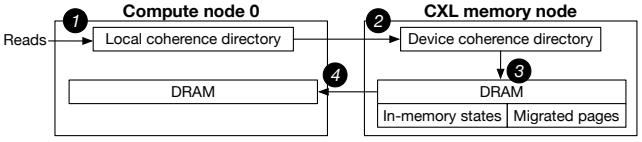


Figure 8. Read workflow of a naive coherence solution

Figure 8 illustrates the workflow of this naive coherence solution. A read access from the owning host to a partially migrated page first queries the local coherence directory to determine whether any caches within the host contain the most recent data ❶. If not found locally (i.e., Invalid state), the CXL device coherence directory is consulted to check whether the caches of other hosts hold the latest version ❷. If this also yields no result, the corresponding 1-bit in-memory state is examined. A value of 0 indicates that the most recent copy resides in CXL memory, prompting data retrieval from

[1]For most server-grade DRAM, each memory line is augmented with additional ECC bits, which are fetched, verified, updated, or discarded together with the data upon every memory access. ECC typically occupies 8 bytes per line, providing several tens of spare bits [27]. These bits have been leveraged as indices for memory remapping (e.g., Intel Flat Mode [65, 99]) or as in-memory states for maintaining NUMA cache coherence (e.g., Intel ccNUMA [52]).

there ❸; conversely, a value of 1 indicates that the latest copy is stored in local memory, leading to a retrieval from local memory ❹. Regardless of the initiating host or the location of the latest data for partially migrated pages, these steps must be executed, incurring unnecessary CXL link round trips that negate the benefits of page migration for local accesses.

4.3.2 PIPM Coherence State Design.

The objective of PIPM coherence design is to ensure that all local accesses to a partially migrated page first query the local memory for the latest data before forwarding requests to the CXL memory node, and to enable incremental migration based on the most recent accessor (i.e., migrate to local DRAM if the most recent accessor is the local host, migrate back to CXL-DSM upon an inter-host access). To accomplish this, we redesign the coherence protocol and utilize 1-bit in-memory states in both local and CXL memory for partially migrated pages.

Extra States. Existing coherence protocols define M, S, and I states—representing Modified, Shared, and Invalid states—in both local coherence directories and device coherence directories. To realize our PIPM coherence design, we introduce an additional per-cache-block in-memory bit in both local and CXL memory, along with a new coherence state (**ME**) in the local coherence directory. By default, the in-memory bit is initialized to 0. When a cache block migrates to local DRAM, this bit is set to 1 in both the local DRAM and CXL-DSM. The coherence directory state combined with the in-memory bit collectively defines the PIPM coherence state of a cache block.

In the local coherence directory, the newly introduced **ME state** (Migrated-Modified/Exclusive) indicates that the corresponding cache block has been migrated to the local memory of the host and is cached exclusively in this host's cache. Subsequent local accesses to cache blocks in the ME state can proceed without querying the device coherence directory, thus enabling efficient coherence handling. The encoding for the ME state comprises a new ME state in the local coherence directory paired with an in-memory bit set to 1, as illustrated in the upper table of Figure 9. Additionally, we introduce the **I' state** (Migrated-Invalid), representing that the cache block is migrated to the local memory of the host but not cached (i.e., Invalid in the directory). The encoding for the I' state reuses the invalid (I) state in the local coherence directory combined with an in-memory bit set to 1, as depicted in the upper table of Figure 9.

In the device coherence directory, we also introduce the **I' state** to indicate that the corresponding cache block has been migrated to a host's local memory. Inter-host accesses to cache blocks marked as I' in the device coherence directory must be directed to the host's local memory. The encoding of the I' state reuses the Invalid (I) state in the device coherence directory in conjunction with an in-memory bit set to 1, as illustrated in the lower table of Figure 9.

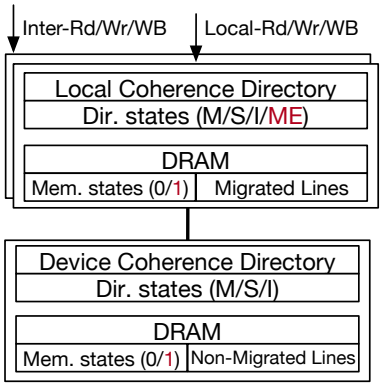
4.3.3 PIPM Coherence State Transition.

The right side of Figure 9 illustrates the PIPM coherence state transitions triggered by various events, including six newly introduced transitions: local writeback operations (case ❶) that initiate incremental migration from CXL memory to a host's local memory; inter-host reads and writes (cases ❷, ❺, and ❻) that trigger incremental migration from local memory back to CXL memory; and efficient local memory accesses (cases ❸ and ❹). For clarity and simplicity, the standard coherence request handling workflow [52, 77], which remains unchanged, is omitted from the following description.

Case ❶: Incremental Migration upon Local WriteBack (`Loc-WB`). When the local directory state is M, it indicates that the local node was the most recent accessor of the cache block (otherwise, the state would be either S or I) and that the block has not yet been migrated into local memory (otherwise, it would be ME). Under this condition, a writeback operation triggers incremental migration. This migration process involves invalidating the corresponding entries in both the host and CXL coherence directories as well as the host's cache entry, retrieving and flipping the associated in-memory state bits in both local and CXL memory, and subsequently performing the incremental migration. Upon completion, the coherence state transitions from M to I' in both the local host directory and the CXL device directory.

Case ❸ and ❹: Local Accesses (`Loc-Rd/Loc-Wr/Loc-WB`) to Migrated Cache Blocks. Once a cache block has been migrated to local memory, ❸ subsequent local memory requests are served directly from local memory, with the host coherence directory updated accordingly (transitioning from I' to ME). Consequently, the CXL directory no longer needs to allocate an entry for this cache block, thereby eliminating unnecessary host-device CXL traffic. ❹ When this cache block is subsequently evicted from the local cache (transitioning from ME back to I'), only a dirty data writeback and invalidation of the corresponding host directory entry are required.

Case ❷: Migration back to CXL-DSM upon inter-host memory accesses (`Inter-Rd/Inter-Wr`) in I' State. When no valid cache copies exist (i.e., the migrated cache block is in the I' state on both the host and device sides), another host's CXL memory access to the migrated line is directed to the CXL device directory. The CXL directory issues a CXL memory read to verify the I' coherence state, after which the request is forwarded to the local directory of the host currently owning the migrated data. The migrated host's local directory retrieves both the memory line and the associated in-memory bit, then performs an asynchronous memory writeback, updating its coherence state from I' to I. Upon receiving this response, the CXL directory allocates a directory



Local Coherence States

Host	Dir. State	Mem. State	PIPM State
Default	M/S/I	0	M/S/I
Extra	ME/I	1	ME/I'

Extra State I' Description: Migrated to local memory; the latest version is not in local cache.

Extra State ME Description: Migrated to local memory, exclusively cached in local cache

Device Coherence State

Device	Dir. State	Mem. State	PIPM State
Default	M/S/I	0	M/S/I
Extra	I	1	I'

Extra State I' Description: Migrated to local memory

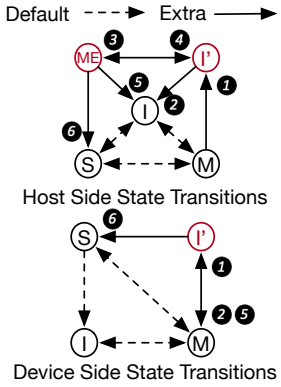


Figure 9. PIPM coherence design.

entry for the cache line and updates its state to M. Finally, the retrieved data is cached in the requester host's cache.

Case ❺ and ❻: Migration back to CXL memory upon inter-host accesses (`Inter-Rd/Inter-Wr`) in ME state. When a migrated cache line is exclusively cached at the local host (i.e., in the ME state on the host side and the I' state on the device side), inter-host accesses are still routed through the requester host's directory, the CXL directory, and finally the owning host's local directory. The owning host's local directory subsequently updates its coherence state—transitioning from ME to I for ❺ `Inter-Wr`, or from ME to S for ❻ `Inter-Rd`—and initiates an asynchronous memory writeback to update the in-memory state bit. Upon receiving the response, the CXL directory allocates an entry and updates its coherence state accordingly: from I to M for case ❺, or from I to S for case ❻. Finally, the requested data is cached in the requester host's cache.

Interaction with global and local remapping tables. PIPM requires accessing global and local remapping tables only for shared data access. For local private data (i.e., data allocated and pinned in local DRAM for security or performance considerations) access, PIPM does not introduce any remapping table lookups or coherence request handling modifications. When initiating a memory request, existing processors that support CXL first perform a simple physical address range check to route the memory request to the local memory controller or the CXL RC accordingly. As accesses to shared data always carry physical addresses within the CXL-DSM physical address range *regardless of whether the shared data pages are partially migrated or not* after virtual-to-physical address translation and before remapping table lookup, processors can always distinguish local private data accesses from shared data accesses after the range check.

For shared data access, on each LLC miss (i.e., when the local coherence directory is in I state), the requester needs to first perform a local remapping table lookup to retrieve the full local coherence state (I or I'). Also, each migrated memory line access requires a local remapping table lookup. Global remapping table access occurs only when forwarding remote access requests (case ❷, ❺ and ❻).

PIPM does not introduce extra CXL directory resource contention beyond default CXL-DSM but instead reduces it, as migrated cache lines no longer require CXL directory entry allocation.

4.4 Space Overhead

The local remapping table on each host's DRAM requires 4 Bytes per entry to store a 28-bit PFN (capable of indexing up to 1TB of local DRAM) and a 4-bit access counter. It is organized as a two-level radix page table [63, 80] with a fixed root node size of 32MB (8 Bytes per entry, indexing up to 4M page table pages, where each PT page stores 1K page table entries) to balance access latency and storage overhead. It requires only (32MB + 4B/4KB × RSS), which is approximately 0.1% of the total resident Set Size (RSS) of the workloads. The global remapping table in CXL-DSM requires only 2 Bytes per entry (consisting of a 5-bit current ID, a 5-bit candidate ID, and a 6-bit access counter), accounting for just 0.05% of the total CXL-DSM size. By default, PIPM requires only a 16KB global remapping cache on the CXL device and a 1MB local remapping cache on each host's RC to effectively cache remapping entries.

4.5 Discussion

Majority-Vote Generality and Scalability. Our majority-vote mechanism is lightweight and access-driven, allowing it to generalize across diverse workload behaviors without relying on workload-specific heuristics. When access patterns are short-term-balanced across hosts, the design correctly avoids migration and retains data in the CXL memory, preventing unnecessary movement. As the host count increases, the majority-vote approach continues to suppress performance-degrading migrations and consistently outperforms prior designs. Moreover, PIPM's implementation as a

system-wide hardware mechanism ensures inherent scalability independent of software configurations (e.g., VM count).

5 Evaluation

5.1 Evaluation Methodology

5.1.1 Benchmarks. Our target large-scale multi-host systems typically run memory-intensive workloads with large memory footprints that do not fit within a single socket and large working set sizes that significantly exceed on-die LLC capacities. Following prior work [15–17, 28, 66], we select representative large-scale, memory-intensive workloads, as listed in Table 1.

Table 1. Evaluated workloads.

Benchmark	Benchmark Suite	Memory Footprint
SSSP (Single-Source Shortest Paths)	GAPBS [9] (Kron)	48GB
BFS (Breadth-first Search)	GAPBS	48GB
PR (Compute the PageRank score)	GAPBS	48GB
CC (Connected components)	GAPBS	48GB
BC (Betweenness centrality)	GAPBS	48GB
TC (Triangle Counting)	GAPBS	48GB
XSBench (Computational kernel of the Monte Carlo neutron transport algorithm)	XSBench [81]	42GB
streamcluster (Data stream clustering)	PARSEC [98]	18GB
fluidanimate (Fluid simulation)	PARSEC	10GB
canneal (Annealing simulation)	PARSEC	12GB
bodytrack (Annealed particle filter)	PARSEC	8GB
TPC-C (Default) (Transaction)	Silo [84]	24GB
YCSB (R:W 4:1) (Database)	Silo	15GB

5.1.2 Simulation Methodology. We model the multi-host CXL-DSM architecture using a cycle-level, trace-based timing simulator [1, 24]. The simulator configuration is detailed in Table 2. Following prior works [15, 89], our simulation methodology consists of the following steps: (1) We first execute the target multi-threaded workloads on real hardware and use Intel Pintool [8] to collect instruction and memory traces for each thread. (2) We then replay the collected memory traces on the simulator to generate memory mapping checkpoints at every 1-billion-instruction interval. (3) Finally, we perform detailed core simulation, beginning after a warm-up phase, utilizing the corresponding checkpoints and traces. This methodology enables the simulation of applications with memory footprints on the order of tens of gigabytes and sufficiently long runtime (10 billion instructions per core).

5.1.3 Compared Schemes. We compare PIPM against the following related works: (1) **Native CXL-DSM**, the baseline configuration that does not support data migration to hosts' local memory; (2) **Nomad**[90], which employs a state-of-the-art recency-based hotness migration policy[34, 55] and optimizes kernel-based page migration by enabling asynchronous migration; (3) **Memtis**[45], utilizing a state-of-the-art frequency-based hotness migration policy; and (4) **HeMem**[68], another frequency-based hotness migration method. We also introduce two ablation baselines to separately analyze the effectiveness of PIPM's migration policy and mechanism: (5) **OS-skew**, which combines the PIPM migration policy with a conventional kernel-based migration mechanism; and (6) **HW-static**, which employs incremental migration enabled by the PIPM coherence protocol but with a static mapping strategy (i.e., without our adaptive migration policy), analogous to prior hardware-tiering approaches such as Intel Flat Mode [65, 99]. Under HW-static, CXL-DSM is uniformly partitioned and statically mapped to each host's local memory. We also include an upper-bound estimation, (7) **Local-only**, where the workloads run on a single-socket CPU with sufficiently large DRAM to hold all data.

Table 2. Scaled-down System Configuration.

Architecture	4 hosts, 1 single-socket CPU each host
CPU	4 OoO cores, 4GHz, 6-wide, 224-entry ROB, 72-entry LQ, 56-entry SQ
Private L1-(I/D)	32KB, 8-way, 4 cycle RT (round-trip) latency
Shared LLC	2MB per core, 16-way, 24-cycle RT latency
DRAM	2x DDR5-4800 channels 128GB CXL-DSM; 1x DDR5-4800 channel 32GB DRAM per host
tRC-tRCD-tCL-tRP	48-15-20-15
CXL link	latency: 50ns, bandwidth: 5GB/s (per direction)
CXL Directory	2048-set, 16-way per slice, 16 slices, 32-cycle RT latency, 2GHz
PIPM parameters	16KB 8-way global remapping cache, 4-cycle RT; 1MB 8-way local remapping cache, 8-cycle RT; Migration threshold: 8

5.1.4 Correctness and Implementation. We implement the PIPM cache coherence protocol on top of the MSI protocol and verify it using the model checking tool Murφ [22], proving that PIPM coherence does not incur any deadlock, and does not violate conceived Single-Writer-Multiple-Reader (SWMR) invariant and Sequential Consistency (SC) model. For simulation, we implement packet-level coherence behaviors for both default CXL-DSM and the PIPM coherence protocol using a locked-based scheme similar to ZSim [73]'s implementation. Based on this, we are able to model full system cache coherency including per-core private cache, and both on-chip and off-chip network traffic. For all evaluation, we assume the code segment, kernel components (e.g., page tables), and thread stacks are treated as private local data, while heap data (e.g., database instances, graphs) are shared across hosts. Following prior work about multi-host CXL-DSM [6, 33, 94], we initially place all shared data in CXL-DSM.

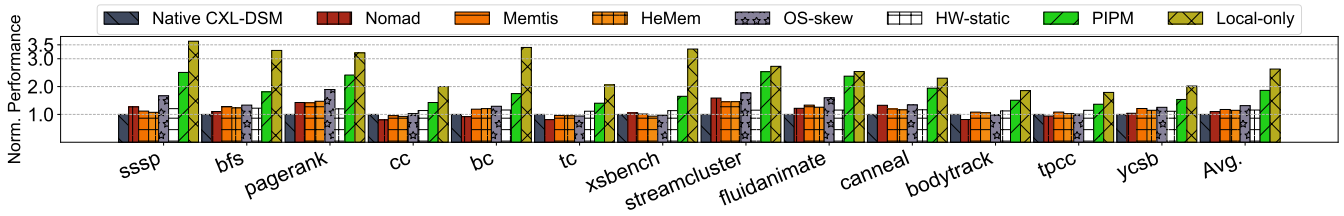


Figure 10. End-to-end performance normalized to Native CXL-DSM.

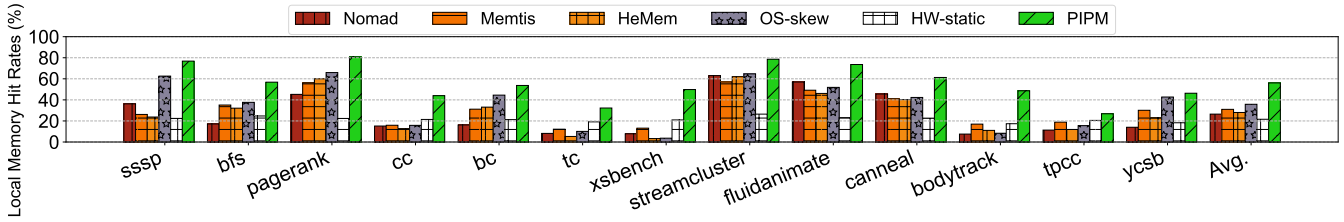


Figure 11. Local memory hit rates.

For page migration schemes, we assume a 20μs 4KB migration-induced overhead for the initiating core [56, 93], a 5μs overhead for other cores, a 10ms migration interval [68] and apply optimizations such as batching TLB shootdowns [30, 31] and multi-threaded, batched page transfers [93] to reduce page migration overhead. For PIPM, migration decisions are made immediately upon exceeding the promotion threshold, as it incurs no kernel-induced overhead or whole-page transfers. We empirically set migration thresholds for both PIPM (where we observe similar performance with threshold ranging from 4 to 16) and baseline schemes for the best performance.

5.2 End-to-end Performance

5.2.1 Overall Performance. Figure 10 presents the overall performance of all evaluated schemes normalized to the *Native CXL-DSM* baseline. PIPM outperforms the other schemes across all workloads, achieving an average performance of 1.86× and 0.73× (up to 2.54× and 0.94×) compared to *Native CXL-DSM* and *Ideal*, respectively, underscoring its substantial performance benefits. Specifically, graph analytics workloads such as SSSP and PageRank, where worker threads independently access memory with strong locality patterns (e.g., adjacency matrix traversals), demonstrate significant performance improvements ranging from 142% to 151%. Database workloads such as TPC-C and YCSB, characterized by random and scattered user-thread accesses, yield more modest performance gains (36%–53%). In contrast, existing page migration schemes employing traditional hotness-based policies (*Nomad*, *Memtis*, and *HeMem*) achieve only marginal improvements over *Native CXL-DSM* and even degrade performance by up to 18% in five workloads. This inefficiency arises because these single-host-oriented designs neither account for migration-induced side effects nor optimize migration overhead, significantly restricting performance potential of page migration in multi-host CXL-DSM scenarios.

5.2.2 Ablation. The *OS-skew* baseline, despite employing the PIPM migration policy, achieves only a 31.5% average improvement over *Native CXL-DSM* due to its inefficient and rigid page-migration mechanism. The *HW-static* baseline leverages hardware-based incremental cache block migration via the PIPM coherence protocol but employs a fixed, static mapping between CXL-DSM and each host's local memory. Consequently, data blocks benefiting from local caching may be inefficiently mapped into other hosts' memory, substantially limiting potential performance gains from fine-grained migration. As a result, *HW-static* yields a modest average improvement of only 15.7% over *Native CXL-DSM*. Overall, PIPM surpasses both *OS-skew* and *HW-static* by an average of 41.7% and 61.1%, respectively. These results demonstrate that both the partial incremental migration mechanisms and the PIPM migration policy are critical for achieving effective memory management for multi-host CXL-DSM systems.

5.3 Performance Analysis

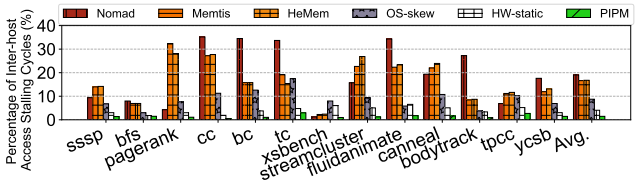


Figure 12. Stalling cycles of inter-host memory access normalized to native CXL-DSM total execution time.

5.3.1 Memory Access Characteristics. To further investigate the effectiveness of PIPM's migration mechanism, we evaluate both the local memory hit ratio and the contribution of inter-host memory access stalls to the total execution time for all schemes.

Figure 11 presents the local memory hit rates across all schemes, where misses are directed to either CXL memory or another host's memory. PIPM achieves a local memory hit rate of 56.1% on average, significantly outperforming *Nomad* (26.5%), *Memtis* (31.0%), and *HeMem* (28.1%). *OS-skew* exhibits a relatively higher local hit rate due to its use of the PIPM migration policy.

Figure 12 illustrates the contribution of stalling cycles from inter-host memory accesses to overall execution time. *Nomad*, *Memtis*, and *HeMem* incur higher stall contributions (averaging 19.1%, 16.6%, and 16.8%, respectively) due to their whole-page migration strategies, which hinder rapid data migration between host memory and CXL memory, thus increasing inter-host memory access frequency. *OS-skew* achieves lower stall contributions from inter-host memory accesses (8.7% on average) owing to the PIPM migration policy, which effectively prevents migration of pages into a host's memory when there are frequent accesses from other hosts.

HW-static induces fewer inter-host memory accesses than kernel-based baselines, contributing only 4.1% to total execution time. However, as shown in Figure 11, it also results in a lower local memory access ratio (21.6% on average), due to its inability to dynamically remap data to hosts that could better utilize local memory. In contrast, PIPM demonstrates the lowest inter-host memory access stall overhead (only 1.5% of total execution time) while simultaneously maintaining the highest local memory access ratio (56.1% on average).

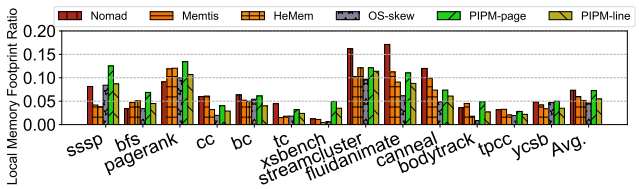


Figure 13. Average ratios of local memory footprint per host to total memory footprint.

5.3.2 Memory Consumption. Figure 13 illustrates the average ratios of local memory footprint per host to the total memory footprint. Traditional hotness-based migration policies (*Nomad*, *HeMem*, and *Memtis*) migrate frequently accessed pages into local memory without considering inter-host memory access, resulting in average per-host memory allocations of 7.4%, 6.0%, and 5.2%, respectively. In contrast, *OS-skew* selectively migrates pages to local memory, thereby reducing its average per-host allocation to 4.6%. The *HW-static* baseline employs a static 1:1 mapping strategy (similar to Intel Flat Mode [65, 99]), lacking dynamic remapping capability and thus maintaining a fixed local memory allocation of 25% per host. In comparison, PIPM leverages both its migration policy and partial incremental migration mechanism, allocating an average of 7.3% of the total memory footprint at the page level, while performing actual cache line migration for 5.5% of the total footprint, as shown in *PIPM-page* and *PIPM-line*, respectively.

5.4 Sensitivity Study and Scalability

5.4.1 Sensitivity to CXL Link Latency. Figure 14 shows the relative performance improvement of PIPM over *Native CXL-DSM* under different CXL link latencies. At a higher link latency of 100ns per direction (representative of configurations with a CXL switch), PIPM achieves an additional performance improvement of 55.7% on average (up to 193.1%), as the benefits of local memory access become more pronounced.

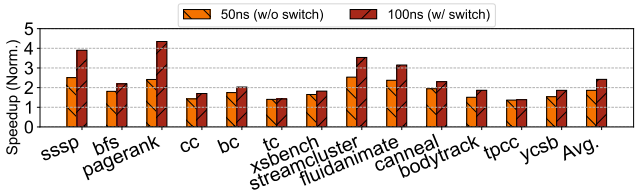


Figure 14. Overall IPC Performance Speedup over Native CXL-DSM under Different CXL Link Latencies.

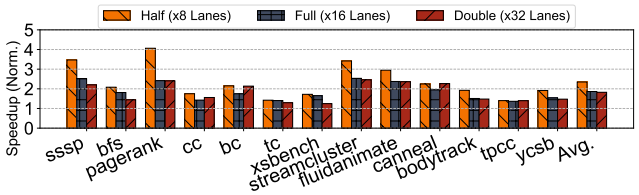


Figure 15. Overall IPC Performance Speedup over Native CXL-DSM under Different CXL Link Bandwidths.

5.4.2 Sensitivity to CXL Link Bandwidth. We use an 8x scaled-down setting as the default configuration (32 cores $\Rightarrow$ 4 cores per host, 64 GB/s (40 GB/s effective [15]) $\Rightarrow$ 8 GB/s (5 GB/s) over x16 CXL lanes). As shown in Figure 15, with half the bandwidth (x8 CXL lanes), PIPM achieves an 48.4% (up to 96%) performance gain over Native CXL-DSM relative to the x16 lanes setting, as most applications become both bandwidth- and latency-bound and thus benefit more from partial incremental migration. With 2x bandwidth (x32 CXL lanes), PIPM retains 97.9% of the relative performance improvement over *Native CXL-DSM* achieved under the x16 lanes setting, demonstrating that most workloads still significantly benefit from partial incremental migration due to their latency-bound characteristics.

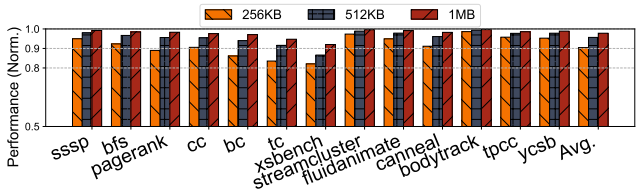


Figure 16. Performance of different local remapping cache Sizes, normalized to infinite local remapping cache size.

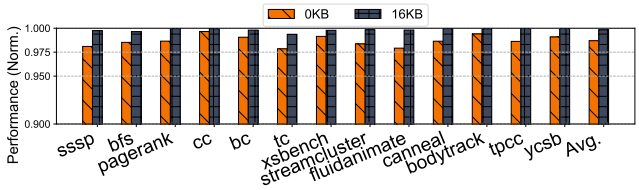


Figure 17. Performance of different global remapping cache sizes, normalized to infinite global remapping cache size.

5.4.3 Sensitivity to Area Overhead.

We vary the on-die buffer capacities of both the local remapping cache and the global remapping cache to evaluate their impact on end-to-end performance. As shown in Figure 16 and Figure 17, the local remapping cache capacity has a higher impact on overall performance, as local remapping table lookups are on the critical path of local memory accesses, whereas global remapping table accesses occur only on inter-host memory accesses. We observe that a 16KB global remapping cache is sufficient to achieve 99.8% of the performance of an ideal infinite global remapping cache, while a 1MB local remapping cache per host achieves 97.8% of the performance of an ideal infinite local remapping cache. *Overall, the area overhead of PIPM is negligible*, requiring only a 1MB local remapping cache per host on the RC, and a 16KB global remapping cache on the CXL device.

6 Related Work

In addition to the related work discussed in Section 2 and Section 3, this section covers other related studies.

Application-level Optimization over CXL-DSM. Recent works [29, 37, 39, 88, 91, 94, 95] focus on application-level optimizations for (CXL-DSM-based) large shared memory pools, including SW prefetching [37, 39], SW-managed coherence [91, 94, 95], replications [91, 95]. *PIPM is orthogonal to these works and can even further support application-level optimizations by exposing software interfaces to programmers.* For example, applications can leverage PIPM's line-level migration to enable fine-grained, lock-free prefetching, or explicitly enable or disable incremental migration for specific pages based on program semantics to improve performance. Also, the PIPM coherence can potentially mitigate the on-die area overhead of the CXL coherence directory [75, 88] for supporting CXL 3.0 multi-host coherence, as migrated memory lines no longer require allocating CXL directory entries until they are migrated back to CXL-DSM.

Automatic Memory Management. A large number of prior works explore page management for tiered memory systems [27, 45, 51, 55, 65, 67–69, 76, 78, 90, 92, 96, 99, 100] and NUMA systems [32, 34]. In contrast, PIPM targets multi-host CXL-DSM systems. PIPM tackles the inefficiency of existing page migration schemes over multi-host CXL-DSM systems by enabling meticulously combining a coherence-aware, incremental migration mechanism with page-level migration policy, PIPM tackles the inefficiency of existing page migration schemes over multi-host CXL-DSM systems while maintaining low overhead.

Distributed Shared Memory Systems. Previous distributed shared memory systems [12, 44, 58, 87] rely on interconnects with socket-like interfaces (e.g., RDMA). They typically employ page-based block granularity and locked-based software cache coherency with manually managed data placement. With the emerging CXL interconnects and hardware cache-coherent CXL-DSM introduced in CXL 3.x, distributed shared memory systems are able to support more efficient, finer-grained data management at rack scale with less software modification. Our work built on top of CXL-DSM proposes architectural support to further unlock the potential of CXL for distributed shared memory systems.

7 Conclusion

We propose **Partial and Incremental Page Migration (PIPM)** for multi-host CXL-DSM, which selectively migrates frequently accessed cache blocks into local memory and incrementally transfers data using intrinsic memory accesses. We develop architectural support including global and local remapping tables, PIPM migration policy, and PIPM coherence protocol to effectively enable partial and incremental page migration. Evaluations show PIPM achieves up to 2.54× (1.86× average) speedup over existing methods, systematically overcoming key limitations of multi-host CXL-DSM.

Acknowledgments

We would like to thank the anonymous reviewers from ASPLOS 2026 for their insightful and constructive feedback, and Jian Zhang, for shepherding our paper. We thank the CRSS IAB members Marvell, Nutanix, ARM, and Cerabyte for their generous support.

References

[1] 2025. ChampSim. https://github.com/ChampSim/ChampSim.
[2] Steve Abraham. 2016. Amazon Aurora Multi-Master: Scaling out database write performance. (2016).

[3] Marcos K Aguilera, Nadav Amit, Irina Calciu, Xavier Deguillard, Jayneel Gandhi, Stanko Novakovic, Arun Ramanathan, Pratap Subrahmanyam, Lalith Suresh, Kiran Tati, et al. 2018. Remote regions: a simple abstraction for remote memory. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. 775–787.

[4] Hooyoung Ahn, Seonyoung Kim, Yoomi Park, Woojong Han, Shinyoung Ahn, Tu Tran, Bharath Ramesh, Hari Subramoni, and Dhabaleswar K Panda. 2024. Mpi allgather utilizing cxl shared memory pool in multi-node computing systems. In *2024 IEEE International Conference on Big Data (BigData)*. IEEE, 332–337.

[5] Hasan Al Maruf and Mosharaf Chowdhury. 2020. Effectively prefetching remote memory with leap. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. 843–857.

[6] Chloe Alverti, Stratos Psomadakis, Burak Ocalan, Shashwat Jaiswal, Tianyin Xu, and Josep Torrellas. 2025. CXLfork: Fast Remote Fork over CXL Fabrics. (2025), 210–226.

[7] Emmanuel Amaro, Christopher Branner-Augmon, Zhihong Luo, Amy Ousterhout, Marcos K Aguilera, Aurojit Panda, Sylvia Ratnasamy, and Scott Shenker. 2020. Can far memory improve job throughput?. In *Proceedings of the Fifteenth European Conference on Computer Systems*. 1–16.

[8] Moshe Bach, Mark Charney, Robert Cohn, Elena Demikhovsky, Tevi Devor, Kim Hazelwood, Aamer Jaleel, Chi-Keung Luk, Gail Lyons, Harish Patil, and Ady Tal. 2010. Analyzing Parallel Programs with PIN. *Computer* 43, 3 (2010), 34–41. doi:10.1109/MC.2010.60

[9] Scott Beamer, Krste Asanović, and David Patterson. 2015. The GAP benchmark suite. *arXiv preprint arXiv:1508.03619* (2015).

[10] blocksandfiles.com. 2024. Intel sees CXL as rack-level disaggregator with Optane connectivity. https://blocksandfiles.com/2021/08/18/intel-sees-cxl-as-rack-level-disaggregator/. Online; accessed Jun, 2024.

[11] Robert S Boyer and J Strother Moore. 1991. MJRTY—a fast majority vote algorithm. In *Automated reasoning: essays in honor of Woody Bledsoe*. Springer, 105–117.

[12] Qingchao Cai, Wentian Guo, Hao Zhang, Divyakant Agrawal, Gang Chen, Beng Chin Ooi, Kian-Lee Tan, Yong Meng Teo, and Sheng Wang. 2018. Efficient distributed memory management with RDMA and caching. *Proceedings of the VLDB Endowment* 11, 11 (2018), 1604–1617.

[13] Irina Calciu, M Talha Imran, Ivan Puddu, Sanidhya Kashyap, Hasan Al Maruf, Onur Mutlu, and Aasheesh Kolli. 2021. Rethinking software runtimes for disaggregated memory. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 79–92.

[14] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM transactions on intelligent systems and technology* 15, 3 (2024), 1–45.

[15] Albert Cho and Alexandros Daglis. 2024. StarNUMA: Mitigating NUMA Challenges with Memory Pooling . In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE Computer Society, Los Alamitos, CA, USA, 997–1012. doi:10.1109/MICRO61859.2024.00077

[16] Chiachen Chou, Aamer Jaleel, and Moinuddin K. Qureshi. 2016. CANDY: Enabling coherent DRAM caches for multi-node systems. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)* (Taipei, Taiwan, 2016-10). IEEE, 1–13. doi:10.1109/MICRO.2016.7783738

[17] Chia Chen Chou, Aamer Jaleel, and Moinuddin K. Qureshi. 2014. CAMEO: A Two-Level Memory Organization with Capacity of Main Memory and Flexibility of Hardware-Managed Cache. In *2014 47th Annual IEEE/ACM International Symposium on Microarchitecture* (2014-12). 1–12. doi:10.1109/MICRO.2014.63 ISSN: 2379-3155.

[18] Brian F Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM symposium on Cloud computing*. 143–154.

[19] CXL. 2024. Compute Express Link. https://computeexpresslink.org/. Online; accessed Jun, 2024.

[20] CXL. 2024. CXL 3.1 Specification. https://computeexpresslink.org/wp-content/uploads/2024/02/CXL-3.1-Specification.pdf. Online; accessed Jun, 2024.

[21] Alex Depoutovitch, Chong Chen, Per-Ake Larson, Jack Ng, Shu Lin, Guanzhu Xiong, Paul Lee, Emad Boctor, Samiao Ren, Lengdong Wu, Yuchen Zhang, and Calvin Sun. 2023. Taurus MM: Bringing Multi-Master to the Cloud. *Proc. VLDB Endow.* 16, 12 (Aug. 2023), 3488–3500. doi:10.14778/3611540.3611542

[22] David L Dill. 1996. The Mur ϕ verification system. In *International Conference on Computer Aided Verification*. Springer, 390–393.

[23] Padmapriya Duraisamy, Wei Xu, Scott Hare, Ravi Rajwar, David Culler, Zhiyi Xu, Jianing Fan, Christopher Kennelly, Bill McCloskey, Danijela Mijailovic, et al. 2023. Towards an adaptable systems architecture for memory tiering at warehouse-scale. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 727–741.

[24] Nathan Gober, Gino Chacon, Lei Wang, Paul V Gratz, Daniel A Jimenez, Elvira Teran, Seth Pugsley, and Jinchun Kim. 2022. The championship simulator: Architectural simulation for education and competition. *arXiv preprint arXiv:2210.14324* (2022).

[25] Juncheng Gu, Youngmoon Lee, Yiwen Zhang, Mosharaf Chowdhury, and Kang G Shin. 2017. Efficient memory disaggregation with infiniswap. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. 649–667.

[26] Zhiyuan Guo, Yizhou Shan, Xuhao Luo, Yutong Huang, and Yiying Zhang. 2022. Clio: A hardware-software co-designed disaggregated memory system. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 417–433.

[27] Mark Hildebrand, Julian T. Angeles, Jason Lowe-Power, and Venkatesh Akella. 2021. A Case Against Hardware Managed DRAM Caches for NVRAM Based Systems. In *2021 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 194–204. doi:10.1109/ISPASS51385.2021.00036

[28] Cheng-Chieh Huang, Rakesh Kumar, Marco Elver, Boris Grot, and Vijay Nagarajan. 2016. C3D: Mitigating the NUMA bottleneck via coherent DRAM caches. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)* (2016-10). 1–12. doi:10.1109/MICRO.2016.7783739

[29] Wentao Huang, Mo Sha, Mian Lu, Yuqiang Chen, Bingsheng He, and Kian-Lee Tan. [n. d.]. Bandwidth Expansion via CXL: A Pathway to Accelerating In-Memory Analytical Processing. ([n. d.]).

[30] Ying Huang. 2023. [PATCH -v5 8/9] migrate_pages: batch flushing TLB. (2023). https://patchew.org/linux/20230213123444.155149-1-ying.huang@intel.com/20230213123444.155149-9-ying.huang@intel.com/

[31] Ying Huang. 2023. [PATCH] mm,unmap: avoid flushing TLB in batch if PTE is inaccessible. (2023). https://lkml.indiana.edu/hypermail/linux/kernel/2304.2/05082.html

[32] Ying Huang. 2024. autonuma: Optimize page placement for memory tiering system - Patchwork. (2024). https://patchwork.kernel.org/project/linux-mm/patch/20201027063217.211096-2-ying.huang@intel.com/

[33] Yibo Huang, Haowei Chen, Newton Ni, Vijay Chidambaram, Dixin Tang, Emmett Witchel, Zhiting Zhu, and Zhipeng Jia. 2025. Tigon: A distributed database for a CXL pod. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25), Boston, MA*.

[34] Ying Huang and Hasan Al Maruf. 2021. https://lwn.net/Articles/876993/. [PATCH 0/5] Transparent Page Placement for Tiered-Memory.

[35] SK Hynix. 2024. SK hynix Develops DDR5 DRAM CXLTM Memory to Expand the CXL Memory Ecosystem. https://news.skhynix.com/sk-hynix-develops-ddr5-dram-cxltm-memory-to-expand-the-cxl-memory-ecosystem/. Online; accessed Jun, 2024.

[36] Sunita Jain, Nagaradhesh Yeleswarapu, Hasan Al Maruf, and Rita Gupta. 2024. Memory sharing with CXL: Hardware and software design approaches. *arXiv preprint arXiv:2404.03245* (2024).

[37] Junhyeok Jang, Hanjin Choi, Hanyeoreum Bae, Seungjun Lee, Miryeong Kwon, and Myoungsoo Jung. 2023. CXL-ANNS: Software-Hardware Collaborative Memory Disaggregation and Computation for Billion-Scale Approximate Nearest Neighbor Search. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, Boston, MA, 585–600. https://www.usenix.org/conference/atc23/presentation/jang

[38] Houxiang Ji, Srikar Vanavasam, Yang Zhou, Qirong Xia, Jinghan Huang, Yifan Yuan, Ren Wang, Pekon Gupta, Bhushan Chitlur, Ipoom Jeong, and Nam Sung Kim. 2024. Demystifying a CXL Type-2 Device: A Heterogeneous Cooperative Computing Perspective. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1504–1517. doi:10.1109/MICRO61859.2024.00110

[39] Changyeon Jo, Hyunik Kim, Hexiang Geng, and Bernhard Egger. 2020. RackMem: A Tailored Caching Layer for Rack Scale Computing. In *Proceedings of the ACM International Conference on Parallel Architectures and Compilation Techniques* (Virtual Event, GA, USA) *(PACT '20)*. Association for Computing Machinery, New York, NY, USA, 467–480. doi:10.1145/3410463.3414643

[40] Sudarsun Kannan, Ada Gavrilovska, Vishal Gupta, and Karsten Schwan. 2017. Heteroos: Os design for heterogeneous memory management in datacenter. In *Proceedings of the 44th Annual International Symposium on Computer Architecture*. 521–534.

[41] Sudarsun Kannan, Yujie Ren, and Abhishek Bhattacharjee. 2021. Klocs: Kernel-level object contexts for heterogeneous memory systems. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 65–78.

[42] Marks Kevin. 2024. CXL for Storage. https://www.snia.org/sites/default/files/SDC/Austin/SNIA-RSDC24-Marks-CXL-for-Storage.pdf.

[43] Andres Lagar-Cavilla, Junwhan Ahn, Suleiman Souhlal, Neha Agarwal, Radoslaw Burny, Shakeel Butt, Jichuan Chang, Ashwin Chaugule, Nan Deng, Junaid Shahid, et al. 2019. Software-defined far memory in warehouse-scale computers. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 317–330.

[44] Seung-seob Lee, Yanpeng Yu, Yupeng Tang, Anurag Khandelwal, Lin Zhong, and Abhishek Bhattacharjee. 2021. Mind: In-network memory management for disaggregated data centers. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. 488–504.

[45] Taehyung Lee, Sumit Kumar Monga, Changwoo Min, and Young Ik Eom. 2023. Memtis: Efficient memory tiering with dynamic page classification and page size determination. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 17–34.

[46] Baptiste Lepers and Willy Zwaenepoel. 2023. Johnny Cache: the End of {DRAM} Cache Conflicts (in Tiered Main Memory Systems). In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 519–534.

[47] Chuanhan Li, Jishen Zhao, and Yuanchao Xu. 2025. Efficient Security Support for CXL Memory through Adaptive Incremental Offloaded (Re-) Encryption. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture®*. 1102–1116.

[48] Huaicheng Li, Daniel S Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, et al. 2023. Pond: Cxl-based memory pooling systems for cloud platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 574–587.

[49] Jinshu Liu, Hamid Hadian, Yuyue Wang, Daniel S Berger, Marie Nguyen, Xun Jian, Sam H Noh, and Huaicheng Li. 2025. Systematic cxl memory characterization and performance analysis at scale. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 1203–1217.

[50] Jinshu Liu, Hamid Hadian, Hanchen Xu, and Huaicheng Li. 2025. Tiered Memory Management Beyond Hotness. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. 731–747.

[51] Jinshu Liu, Hamid Hadian, Hanchen Xu, Huaicheng Li, and Virginia Tech. [n. d.]. Tiered Memory Management Beyond Hotness. ([n. d.]).

[52] Kevin Loughlin, Stefan Saroiu, Alec Wolman, Yatin A Manerkar, and Baris Kasikci. 2022. Moesi-prime: preventing coherence-induced hammering in commodity workloads. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 670–684.

[53] Teng Ma, Zheng Liu, Chengkun Wei, Jialiang Huang, Youwei Zhuo, Haoyu Li, Ning Zhang, Yijin Guan, Dimin Niu, Mingxing Zhang, et al. 2024. {HydraRPC}:{RPC} in the {CXL} Era. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 387–395.

[54] Adnan Maruf, Ashikee Ghosh, Janki Bhimani, Daniel Campello, Andy Rudoff, and Raju Rangaswami. 2022. MULTI-CLOCK: Dynamic Tiering for Hybrid Memory Systems. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 925–937. doi:10.1109/HPCA53966.2022.00072

[55] Hasan Al Maruf, Hao Wang, Abhishek Dhanotia, Johannes Weiner, Niket Agarwal, Pallab Bhattacharya, Chris Petersen, Mosharaf Chowdhury, Shobhit Kanaujia, and Prakash Chauhan. 2023. Tpp: Transparent page placement for cxl-enabled tiered-memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 742–755.

[56] Mitesh R. Meswani, Sergey Blagodurov, David Roberts, John Slice, Mike Ignatowski, and Gabriel H. Loh. 2015. Heterogeneous memory architectures: A HW/SW approach for mixing die-stacked and off-package memories. In *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, Burlingame, CA, USA. doi:10.1109/hpca.2015.7056027

[57] Onur Mutlu. 2013. Memory scaling: A systems architecture perspective. In *2013 5th IEEE International Memory Workshop*. IEEE, 21–25.

[58] Jacob Nelson, Brandon Holt, Brandon Myers, Preston Briggs, Luis Ceze, Simon Kahan, and Mark Oskin. 2015. {Latency-Tolerant} software distributed shared memory. In *2015 USENIX Annual Technical Conference (USENIX ATC 15)*. 291–305.

[59] nextplatform.com. 2024. A coherent interconnect strategy: CXL absorbs Gen-Z. https://www.nextplatform.com/2021/11/23/finally-a-coherent-interconnect-strategy-cxl-absorbs-gen-z/. Online; accessed Jun, 2024.

[60] nextplatform.com. 2024. CXL and Gen-Z Iron Out a Coherent Interconnect Strategy. https://www.nextplatform.com/2020/04/03/cxl-and-gen-z-iron-out-a-coherent-interconnect-strategy/. Online; accessed Jun, 2024.

[61] nextplatform.com. 2024. PCI-Express 5.0: The unintended but formidable datacenter interconnect. https://www.nextplatform.com/2021/02/03/pci-express-5-0-the-unintended-but-formidable-datacenter-interconnect//. Online; accessed Jun, 2024.

[62] Vlad Nitu, Boris Teabe, Alain Tchana, Canturk Isci, and Daniel Hagimont. 2018. Welcome to zombieland: Practical and energy-efficient memory disaggregation in a datacenter. In *Proceedings of the Thirteenth EuroSys Conference*. 1–12.

[63] Chang Hyun Park, Ilias Vougioukas, Andreas Sandberg, and David Black-Schaffer. 2020. Page Tables: Keeping them Flat and Hot (Cached). *arXiv preprint arXiv:2012.05079* (2020).

[64] Christian Pinto, Dimitris Syrivelis, Michele Gazzetti, Panos Koutsovasilis, Andrea Reale, Kostas Katrinis, and H Peter Hofstee. 2020. Thymesisflow: A software-defined, hw/sw co-designed interconnect stack for rack-scale memory disaggregation. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 868–880.

[65] Michael D. Powell, Patrick Fleming, Venkidesh Iyer Krishna, Naveen Lakkakula, Subhiksha Ravisundar, Praveen Mosur, Arijit Biswas, Pradeep Dubey, Kapil Sood, Andrew Cunningham, and Smita Kumar. 2025. Intel Xeon 6 Product Family. *IEEE Micro* 45, 3 (2025), 31–40. doi:10.1109/MM.2025.3553756

[66] Andreas Prodromou, Mitesh Meswani, Nuwan Jayasena, Gabriel Loh, and Dean M. Tullsen. 2017. MemPod: A Clustered Architecture for Efficient and Scalable Migration in Flat Address Space Multi-level Memories. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)* (Austin, TX, 2017-02). IEEE, 433–444. doi:10.1109/HPCA.2017.39

[67] Zhenlin Qi, Shengan Zheng, Ying Huang, Yifeng Hui, Bowen Zhang, Linpeng Huang, and Hong Mei. 2025. Chrono: Meticulous Hotness Measurement and Flexible Page Migration for Memory Tiering. In *Proceedings of the Twentieth European Conference on Computer Systems* (Rotterdam Netherlands, 2025-03-30). ACM, 835–853. doi:10.1145/3689031.3717462

[68] Amanda Raybuck, Tim Stamler, Wei Zhang, Mattan Erez, and Simon Peter. 2021. Hemem: Scalable tiered memory management for big data applications and real nvm. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. 392–407.

[69] Jie Ren, Dong Xu, Junhee Ryu, Kwangsik Shin, Daewoo Kim, and Dong Li. 2024. MTM: Rethinking Memory Profiling and Migration for Multi-Tiered Large Memory. In *Proceedings of the Nineteenth European Conference on Computer Systems*. ACM, Athens Greece, 803–817. doi:10.1145/3627703.3650075

[70] Zhenyuan Ruan, Malte Schwarzkopf, Marcos K Aguilera, and Adam Belay. 2020. {AIFM}:{High-Performance},{Application-Integrated} far memory. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 315–332.

[71] Samsung. 2024. Samsung CXL Solutions – CMM-H. https://semiconductor.samsung.com/us/news-events/tech-blog/samsung-cxl-solutions-cmm-h/. Online; accessed Jun, 2024.

[72] Samsung. 2024. Samsung Unveils Industry-First Memory Module Incorporating New CXL Interconnect Standard. https://news.samsung.com/global/samsung-unveils-industry-first-memory-module-incorporating-new-cxl-interconnect-standard. Online; accessed Jun, 2024.

[73] Daniel Sanchez and Christos Kozyrakis. 2013. ZSim: Fast and accurate microarchitectural simulation of thousand-core systems. *ACM SIGARCH Computer architecture news* 41, 3 (2013), 475–486.

[74] Yizhou Shan, Yutong Huang, Yilun Chen, and Yiying Zhang. 2018. {LegoOS}: A disseminated, distributed {OS} for hardware resource disaggregation. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 69–87.

[75] Debendra Das Sharma. 2023. Compute Express Link (CXL): Enabling Heterogeneous Data-Centric Computing With Heterogeneous Memory Hierarchy. *IEEE Micro* 43, 2 (2023), 99–109. doi:10.1109/MM.2022.3228561

[76] Kevin Song, Jiacheng Yang, Sihang Liu, and Gennady Pekhimenko. 2023. Lightweight Frequency-Based Tiering for CXL Memory Systems. *arXiv preprint arXiv:2312.04789* (2023).

[77] Daniel Sorin, Mark Hill, and David Wood. 2022. *A primer on memory consistency and cache coherence.* Springer Nature.

[78] Yan Sun, Jongyul Kim, Zeduo Yu, Jiyuan Zhang, Siyuan Chai, Michael Jaemin Kim, Hwayong Nam, Jaehyun Park, Eojin Na, Yifan Yuan, Ren Wang, Jung Ho Ahn, Tianyin Xu, and Nam Sung Kim. 2025. M5: Mastering Page Migration and Memory Management for CXL-based Tiered Memory Systems. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Rotterdam, Netherlands) *(ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 604–621. doi:10.1145/3676641.3711999

[79] Yan Sun, Yifan Yuan, Zeduo Yu, Reese Kuper, Chihun Song, Jinghan Huang, Houxiang Ji, Siddharth Agarwal, Jiaqi Lou, Ipoom Jeong, et al. 2023. Demystifying cxl memory with genuine cxl-ready systems and devices. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 105–121.

[80] Cristan Szmajda and Gernot Heiser. 2003. Variable radix page table: A page table for modern architectures. In *Asia-Pacific conference on advances in computer systems architecture*. Springer, 290–304.

[81] John R Tramm, Andrew R Siegel, Tanzima Islam, and Martin Schulz. 2014. XSBench-the development and verification of a performance abstraction for Monte Carlo reactor analysis. *The Role of Reactor Physics toward a Sustainable Future (PHYSOR)* (2014).

[82] Tu Tran, Mustafa Abduljabbar, Hooyoung Ahn, Seonyoung Kim, Yoomi Park, Woojong Han, Shinyoung Ahn, Hari Subramoni, and Dhabaleswar K. Panda. 2024. OMB-CXL: A Micro-Benchmark Suite for Evaluating MPI Communication Utilizing Compute Express Link Memory Devices. In *Practice and Experience in Advanced Research Computing 2024: Human Powered Computing* (Providence, RI, USA) *(PEARC '24)*. Association for Computing Machinery, New York, NY, USA, Article 27, 8 pages. doi:10.1145/3626203.3670533

[83] Chun-Wei Tsai, Chin-Feng Lai, Han-Chieh Chao, and Athanasios V Vasilakos. 2015. Big data analytics: a survey. *Journal of Big data* 2, 1 (2015), 21.

[84] Stephen Tu, Wenting Zheng, Eddie Kohler, Barbara Liskov, and Samuel Madden. 2013. Speedy transactions in multicore in-memory databases. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*. 18–32.

[85] Midhul Vuppalapati and Rachit Agarwal. 2024. Tiered Memory Management: Access Latency is the Key!. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) *(SOSP '24)*. Association for Computing Machinery, New York, NY, USA, 79–94. doi:10.1145/3694715.3695968

[86] Chenxi Wang, Haoran Ma, Shi Liu, Yuanqi Li, Zhenyuan Ruan, Khanh Nguyen, Michael D Bond, Ravi Netravali, Miryung Kim, and Guoqing Harry Xu. 2020. Semeru: A {Memory-Disaggregated} managed runtime. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 261–280.

[87] Qing Wang, Youyou Lu, Erci Xu, Junru Li, Youmin Chen, and Jiwu Shu. 2021. Concordia: Distributed shared memory with {In-Network} cache coherence. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*. 277–292.

[88] Zhao Wang, Yiqi Chen, Cong Li, Dimin Niu, Tianchan Guan, Zhaoyang Du, Xingda Wei, and Guangyu Sun. 2025. Enabling Efficient Transaction Processing on CXL-Based Memory Sharing. arXiv:2502.11046 [cs] doi:10.48550/arXiv.2502.11046

[89] Roland E. Wunderlich, Thomas F. Wenisch, Babak Falsafi, and James C. Hoe. 2003. SMARTS: accelerating microarchitecture simulation via rigorous statistical sampling. *SIGARCH Comput. Archit. News* 31, 2 (May 2003), 84–97. doi:10.1145/871656.859629

[90] Lingfeng Xiang, Zhen Lin, Weishu Deng, Hui Lu, Jia Rao, Yifan Yuan, and Ren Wang. 2024. Nomad:{Non-Exclusive} Memory Tiering via Transactional Page Migration. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 19–35.

[91] Tong Xing and Antonio Barbalace. 2025. Rethinking Applications' Address Space with CXL Shared Memory Pools. In *Proceedings of the 4th Workshop on Heterogeneous Composable and Disaggregated*

Systems (HCDS '25). Association for Computing Machinery, New York, NY, USA, 52–59. doi:10.1145/3723851.3723858

[92] Dong Xu, Junhee Ryu, Kwangsik Shin, Pengfei Su, and Dong Li. 2024. {FlexMem}: Adaptive Page Profiling and Migration for Tiered Memory. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 817–833.

[93] Zi Yan, Daniel Lustig, David Nellans, and Abhishek Bhattacharjee. 2019. Nimble page management for tiered memory systems. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 331–345.

[94] Xinjun Yang, Yingqiang Zhang, Hao Chen, Feifei Li, Gerry Fan, Yang Kong, Bo Wang, Jing Fang, Yuhui Wang, Tao Huang, Wenpu Hu, Jim Kao, and Jianping Jiang. 2025. Unlocking the Potential of CXL for Disaggregated Memory in Cloud-Native Databases. In *Companion of the 2025 International Conference on Management of Data* (Berlin, Germany) *(SIGMOD/PODS '25)*. Association for Computing Machinery, New York, NY, USA, 689–702. doi:10.1145/3722212.3724460

[95] Xinjun Yang, Yingqiang Zhang, Hao Chen, Feifei Li, Bo Wang, Jing Fang, Chuan Sun, and Yuhui Wang. 2024. PolarDB-MP: a multi-primary cloud-native database via disaggregated shared memory. In *Companion of the 2024 International Conference on Management of Data*. 295–308.

[96] Xinyue Yi, Hongchao Du, Yu Wang, Jie Zhang, Qiao Li, and Chun Jason Xue. 2025. ArtMem: Adaptive Migration in Reinforcement Learning-Enabled Tiered Memory. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. ACM, Tokyo Japan, 405–418. doi:10.1145/3695053.3731001

[97] Yanpeng Yu, Nicolai Oswald, and Anurag Khandelwal. 2025. CORD: Low-Latency, Bandwidth-Efficient and Scalable Release Consistency via Directory Ordering. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture* (Tokyo Japan, 2025-06-21). ACM, 1311–1326. doi:10.1145/3695053.3731074

[98] Xusheng Zhan, Yungang Bao, Christian Bienia, and Kai Li. 2017. PARSEC3. 0: A multicore benchmark suite with network stacks and SPLASH-2X. *ACM SIGARCH Computer Architecture News* 44, 5 (2017), 1–16.

[99] Yuhong Zhong, Daniel S Berger, Carl Waldspurger, Ishwar Agarwal, Rajat Agarwal, Frank Hady, Karthik Kumar, Mark D Hill, Mosharaf Chowdhury, and Asaf Cidon. 2024. Managing Memory Tiers with CXL in Virtualized Environments. In *Symposium on Operating Systems Design and Implementation*.

[100] Zhe Zhou, Yiqi Chen, Tao Zhang, Yang Wang, Ran Shu, Shuotao Xu, Peng Cheng, Lei Qu, Yongqiang Xiong, Jie Zhang, et al. 2024. NeoMem: Hardware/Software Co-Design for CXL-Native Memory Tiering. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1518–1531.